

팀이 이해하고 운영할 수 있는 플랫폼

다음 사람이 읽고 변경하기 쉬운 구조를 기준으로, 운영 복잡도와 장애·접근 관리 문제를 개선했다.

프로젝트	이전 문제	핵심 변화
GitOps 재설계	환경·저장소마다 다른 설정 관리	모노레포 + 공통 생성 규칙
EKS 가용성	한 노드에 몰린 replica	배치·재배치·보호·노드 공급 분리
Authentik SSO	도구별 계정과 장기 access key	중앙 인증 + 임시 자격 증명
Istio → Traefik	필요한 기능보다 큰 운영 부담	L7 라우팅 중심으로 단순화

읽는 방법

빠른 검토: 2-5쪽에서 문제·구조·선택 이유를 확인합니다.

상세 검토: 부록에서 설정 예시, 검증 범위와 운영 과제를 확인합니다.

사례	요약	상세 설계
GitOps 재설계	2쪽으로 이동	부록 1 열기
EKS 워크로드 가용성	3쪽으로 이동	부록 2 열기
Authentik SSO	4쪽으로 이동	부록 3 열기
Istio → Traefik	5쪽으로 이동	부록 4 열기

[현재 공개 코드](#) · [이전 배포 저장소](#)

설계 근거와 공개 코드 링크는 문서 안에서 클릭할 수 있습니다.

1. GitOps 재설계 - 공통 규칙과 환경 차이 분리

문제: 환경별 브랜치 동기화와 Terraform Cloud 비용에 더해, 인프라·배포 연결 변경도 두 저장소를 오가야 했다.

01 저장소 개편 · 공통 규칙과 서비스 입력 분리



선택	왜 이 방식인가	구현 핵심
S3 + GitHub Actions	Cloud 비용 · 환경 브랜치 동기화 부담 완화	main + 환경 leaf, PR plan · merge apply
Terramate	팀 가독성 · DRY · 직관적인 확장	globals · import · native .tf 생성
모노레포	한 팀이 연결 변경을 한 PR로 검토	인프라 · 배포 설정을 한 컨텍스트에서 추적
공통 Helm	서비스별 리소스 구조 복제 방지	templates · helpers 공통화, values로 차이 입력

stacks/acme/eks/main/dev = **도메인 → 리소스 → 대상 → 환경**

Terragrunt도 HCL 기반 대안이다. 생성된 Terraform을 직접 읽는 방식이 더 직관적이었으며, Cloud UI와 지속적인 업데이트도 선택을 뒷받침했다.

결과: backend 선언 39곳 → 생성 규칙 1곳. 69개 stack의 S3 backend 공통 관리.

검증 · 한계: 코드 스냅샷으로 구조 확인. 공통 규칙 변경은 여러 서비스에 영향을 주므로 적용 범위 검토가 필요하다.

상세 설계 · 공개 코드

2. EKS 가용성 - replica 수보다 배치 위치

문제: replica가 한 노드에 몰려 노드 장애가 서비스로 전파됐고, 노드 복귀 뒤에도 쓸림이 남았다.

02 워크로드 가용성 · 배치 → 장애 대응 → 재분산

replica 2 예시



Karpenter · AZ별 NodePool로 분산할 노드 공급

2노드 예시 · 마지막 단계는 교정 설계

선택	왜 이 방식인가	구현 핵심
TSC · 배치	쓸림 제한과 장애 중 복귀를 함께 고려	hostname 편차 1 강제 · zone 권고 · Honor
Descheduler · 재배포	실행 중인 Pod는 자동으로 재분산되지 않음	강제 제약 위반 · 대상 namespace만 교정
PDB · 보호	재배치 중 동시 eviction 억제	자발적 중단 시 최소 가용 replica 유지
Karpenter · 노드 공급	Pod를 나눌 노드도 AZ별로 필요	AZ별 NodePool과 pool별 교체 범위 설정

앱별 values로 점진 적용해 **JVM 동시 기동 부하**를 억제했다. Honor는 허용하지 않는 taint 노드를 분산 계산에서 제외한다.

결과: 기존 kind 재현에서 replica 3개 기준 Ignore는 1개 Pending → Honor는 2:1 실행.

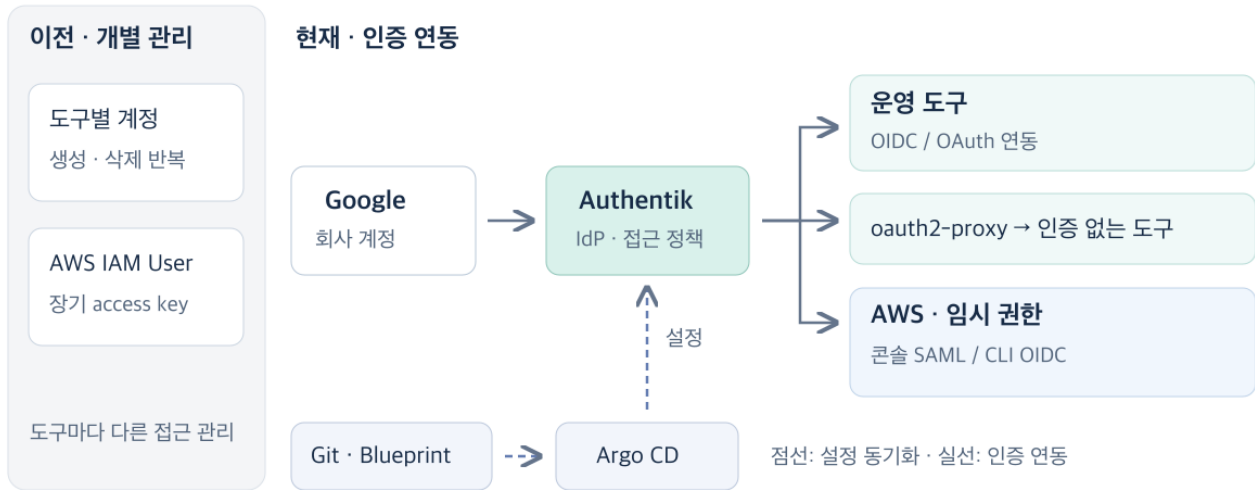
검증 · 한계: Descheduler eviction은 로컬 재현 제외. PDB는 노드 장애를 막지 못하며 생존 노드의 여유 용량이 필요하다.

상세 설계 · 재현 · 공개 코드

3. Authentik SSO - 인증 설정도 Git에서 리뷰

문제: 도구별 계정 생성·회수와 AWS 장기 자격 증명 관리가 분산되어 있었다.

03 SSO · 계정은 통합하고 연동 방식은 앱에 맞게



선택	왜 이 방식인가	구현 핵심
Authentik Blueprint	YAML 설정 관리가 기존 GitOps에 적합	앱 · provider · 정책을 Git에서 리뷰
앱별 연동 방식	앱이 가진 인증 기능을 우선 활용	OIDC · OAuth 직접 연동, 미지원은 proxy
AWS 임시 자격 증명	장기 key 제거 · CLI 반복 로그인 완화	콘솔 SAML, CLI OIDC · PKCE · token 갱신
시크릿 · 권한 분리	secret 비공개와 최소 권한 유지	Secrets Manager → ESO, 그룹별 접근

사용자당 SaaS 요금 대신 **self-host 운영 비용**을 감수했다. Keycloak도 코드 관리가 가능하며, 이 환경에서는 Blueprint 방식이 더 직접적이었다.

결과: 도구별 계정을 중앙 인증으로 통합하고, AWS 장기 key를 임시 자격 증명으로 전환.

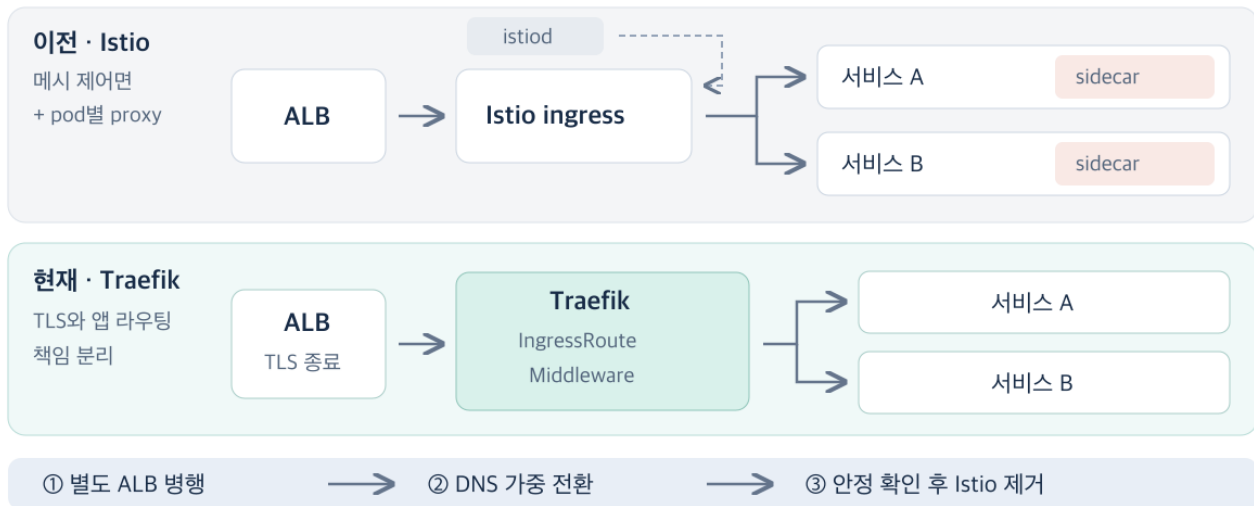
검증 · 과제: Argo CD 정상 상태만으로 적응을 판단하지 않고 Blueprint 내부 상태를 확인한다. 실패 자동 감시와 IdP 가용성 관리가 필요하다.

상세 설계 · 운영 이슈 · 공개 코드

4. Istio → Traefik - 실제 쓰는 기능에 맞추기

문제: 실제 요구는 L7 인그레스였지만, sidecar·Istio·Envoy까지 운영하고 인수인계해야 했다.

04 라우팅 단순화 · 필요한 인그레스 기능에 집중



선택	왜 이 방식인가	구현 핵심
Traefik	인그레스 중심 요구·운영 부담 축소	IngressRoute + 재사용 Middleware
유지보수 전망 반영	전환 당시 ingress-nginx retirement 공지	단기간 내 재전환할 가능성 회피
ALB·Traefik 역할 분리	AWS와 앱 경로 변경을 따로 관리	ALB는 TLS, Traefik은 앱 라우팅
DNS 단계 전환	새 경로 문제 발생 시 되돌릴 수 있어야 함	기존·신규 ALB 병행, DNS 가중치 조정

결과: 20여 개 서비스 전환, sidecar 주입·Istio 제어면 구성 제거.

검증·한계: 경로 기능·요청·지연·에러율을 관찰하며 전환. 서비스 간 mTLS가 필요해지면 선택을 재검토한다.

상세 설계·전환 절차·공개 코드

GitOps 저장소 아키텍처와 공통화 설계

이 문서는 입사 시점의 Terraform 저장소와, 분리되어 있던 Kubernetes manifest 저장소가 현재의 GitOps 모노레포로 수렴한 설계 흐름과, 인프라·애플리케이션 배포 설정을 어떻게 나누고 다시 조합하는지 설명한다. 고정된 코드 스냅샷의 구조를 비교하며, 해당 리소스의 현재 실행 상태를 증명하는 문서는 아니다.

구분	저장소	담당 영역	기준 스냅샷
입사 시점	infra-config-portfolio	Terraform - ECS·RDS·network, Terraform Cloud 실행	83a3c40
개편 전	manifest-k8s-cluster-portfolio	EKS 서비스별 Helm chart·raw manifest	ed4f0a3
현재	devops-configs-portfolio	인프라와 배포 선언을 묶은 모노레포	7e3427a

1. 두 갈래의 이전 저장소에서 GitOps 모노레포로

현재 모노레포는 두 저장소의 후신이다. 인프라 쪽은 ECS 시절의 Terraform 저장소에서, 배포 쪽은 EKS 서비스별 manifest 저장소에서 왔다. 두 저장소는 Terraform용과 manifest용으로 나란히 운영되었고, 입사 후 약 2년간 그 구조를 쓴 뒤 하나로 개편했다.



입사 시점 저장소의 README.md는 “현재는 terraform 코드만 있지만, 향후 Kubernetes와 관련된 선언형 설정 코드들이 이 저장소에 추가될 수 있습니다”라고 적고 있다. 현재 구조는 그 방향을 역할별 폴더로 구체화한 결과다.

1-1. 인프라 쪽: 입사 시점의 Terraform 저장소

하나의 terraform/ 아래에 세 가지 경로 규칙이 공존했다.

terraform/		
├─ network/{dev,prod}/	# <resource>/<env>	환경별 디렉터리 복제
├─ security/{dev,prod}/		
├─ {dev,stage,prod}/common/	# <env>/common/<resource>	환경이 최상위
│ └─ alb/ ecr/ ecs/ s3/		
├─ hospital/<service>/	# <product>/<service>	환경 디렉터리 없음
├─ mall/{back,front}/		
└─ modules/ecs/{single_task,multi_task,v2-ecs,rabbitmq}/		

세 번째 규칙에서는 환경이 경로에 없다. Terraform Cloud workspace 이름에서 환경을 뽑아내고, 환경별 값은 locals의 map으로 들고 있다. hospital/back-v2의 관련 필드 발췌:

```
# version.tf
backend "remote" {
  organization = "acme"
  workspaces { prefix = "hospital-back-v2-" }
}

# local.tf
env      = regex(join("|", values(var.environment)), terraform.workspace)
cpu      = { dev = 512, stage = 512, prod = 512 }
remote_env = { dev = "dev", stage = "prod", prod = "prod" } # stage 전용 network·security 가 없어 p
rod 를 참조
```

즉 “이 stack의 환경이 무엇인가”의 답이 디렉터리 이름, 최상위 폴더, workspace 이름 중 어디에 있는지가 stack마다 달랐다.

관점	입사 시점 (infra-config)	현재 (devops-configs)
워크로드 런타임	ECS Fargate. task definition과 컨테이너 command까지 Terraform이 소유 (backend.tf)	EKS. Terraform은 플랫폼까지, workload는 Helm values와 Argo CD
실행과 state	Terraform Cloud VCS-driven, workspace별 state	GitHub Actions + S3 backend, --changed로 대상 stack 선택
브랜치 모델	dev·stage·prod 환경 브랜치, push마다 Terraform Cloud 트리거	main + feature 브랜치, 환경은 stack tag로 선택
backend 선언	디렉터리마다 직접 쓴 backend "remote" 39 개	imports/backend.tm.hcl 생성 규칙 1개로 69 개 stack의 S3 backend 관리
버전 고정	디렉터리별 선언. Terraform 제약 5종, AWS provider 제약 7종(major 3·4 공존)	루트 config.tm.hcl의 globals 한 곳
환경 차이 표현	디렉터리 복제 또는 locals의 환경 키 map. network/dev/main.tf와 network/prod/main.tf는 158줄 중 Environment 태그 1줄만 다름	leaf에는 stack.tm.hcl·tfvars.tm.hcl 등 차이만 두고 나머지는 생성
stack 간 값 전달	terraform_remote_state 59곳이 workspace 이름 문자열을 참조	input 블록 86개가 stack ID를 참조. terraform_remote_state는 0
CI 자격 증명	workspace 환경 변수에 IAM user access key (terraform.md)	GitHub OIDC로 role assume (_bootstrap/oidc_aws_github)

수치는 두 스냅샷의 선언을 집계한 값이다. 입사 시점의 remote backend는 실험용 terraform/test/를 제외하면 39개, 포함하면 44개다. 현재 전체 75개 stack 중 69개에는 S3 backend, _bootstrap 4개에는 local backend가 있고, pod-identity-agent/kubecost의 dev·prd 2개에는 backend 파일이 없다. 3절의 경로 축, 4절의 생성 규칙과 값 공유는 각각 이 표의 행 하나에 대한 현재의 답이다.

1-2. 배포 쪽: Kubernetes manifest 분리 저장소

개편 전에는 Kubernetes 배포 구성이 별도 저장소에 있었다. 현재 구조는 Terraform/Terramate 인프라, Argo CD Application, Helm chart와 values, raw manifest를 하나의 GitOps 모노레포 안에서 역할별로 재배포한다.

이 변화는 Helm이나 Argo CD를 새로 도입한 것이 아니다. 이전 저장소의 acmemall-backend-v4/README.md에도 Argo CD 기반 흐름이 명시되어 있고, 서비스별 Helm chart와 raw manifest·Kustomize 구성이 함께 존재한다. 개편의 중심은 도

구 교체보다 저장소 경계와 공통화 단위를 다시 설계한 데 있다.

관점	개편 전	현재
저장소 경계	Kubernetes manifest 전용 분리 저장소	인프라와 배포 선언을 묶은 GitOps 모노레포
루트 구조	서비스 이름이 최상위 경계를 형성	stacks, modules, charts, values, argocd, manifests 등 역할별 폴더
Helm chart	api, api-admin 등 서비스 하위에 chart 배치	여러 앱이 charts/app 공통 chart를 사용
환경 values	서비스 chart 옆 values/dev.yaml 등	values/apps/<group>/<version>/<role>/<env>.yaml 축으로 분리
배포 연결	서비스 로컬 Helmfile, Argo CD, raw/Kustomize 구성이 공존	커밋된 Application이 chart·values 또는 manifest를 명시적으로 연결

개편 전 구조에서는 서비스 디렉터리가 배포 단위와 공통화 경계를 함께 나타냈다.

```
acmemall-backend-v4/
├── api/
│   ├── helm/           # API 전용 chart와 templates
│   ├── values/         # dev/prod/stage values
│   └── helmfile.yaml    # 서비스 로컬 chart·values 연결
├── api-admin/
│   ├── helm/
│   └── values/
└── api-notification/
    ├── helm/
    └── values/
```

예를 들어 api/helmfile.yaml은 ./helm과 ./values/{{.Environment.Name}}.yaml을 연결한다. API와 API Admin은 각각 자신의 Chart.yaml과 template 디렉터리를 가진다.

동시에 모든 서비스가 Helm으로 통일된 구조도 아니었다. acme-recommendation-api/dev/kustomization.yaml은 같은 환경 폴더의 Namespace, Deployment, Service를 Kustomize resource로 조합한다.

현재 구조는 서비스별 설정을 없애기보다 위치와 책임을 분리한다. 공통 workload 골격은 charts/app, 서비스·환경 차이는 values/apps, 배포 연결은 argocd, 클라우드 기반은 stacks와 modules가 맡는다. 이후 절에서는 이 현재 구조를 각 계층별로 설명한다.

1-3. 두 저장소를 하나로 합친 이유

분리는 그 자체로 비용이다. 이 규모에서는 그 비용을 정당화할 이점이 없다고 판단했다.

이유	설명
하나의 개념이 두 저장소에 걸친다	인프라와 배포는 이름으로 이어져 있다. 저장소가 나뉘면 그 연결이 PR 두 개와 맞춰야 할 순서로 바뀐다
LLM 보조 작업	저장소별로 나뉜 작업 맥락에서 연결을 추적하려면 다른 저장소를 추가로 탐색해야 했다. 통합해 탐색과 변경 검토를 한 작업 맥락·PR에 모았다
코드 규모	저장소 분리의 이점(독립 권한, 독립 릴리스 주기)은 규모가 클 때 나온다. 한 팀이 관리하는 규모에서는 오가는 비용만 남는다

첫 번째 이유의 예가 컨테이너 이미지 저장소 이름이다. 현재 저장소에서는 같은 문자열이 두 계층에 나란히 있다.

```
stacks/acme/ecr/mall/dev/main.tf      "acmemall-backend-v4/dev/api"      # ECR repository 생성
values/apps/mall/v4/api/dev.yaml      repository: acmemall-backend-v4/dev/api  # 그 이미지를 배포
```

ECR stack과 앱 values가 한 저장소에 있으므로 한 번의 검색으로 양쪽이 나오고, 이름을 바꾸는 변경이 한 PR에 담긴다.

두 번째 이유는 저장소에 흔적이 남아 있다. 루트의 AGENTS.md는 “Terraform 적용 규칙”과 “ArgoCD/Helm 작업 규칙”을 한 파일에 담아, 인프라와 배포의 작업 규칙을 한곳에서 찾도록 한다.

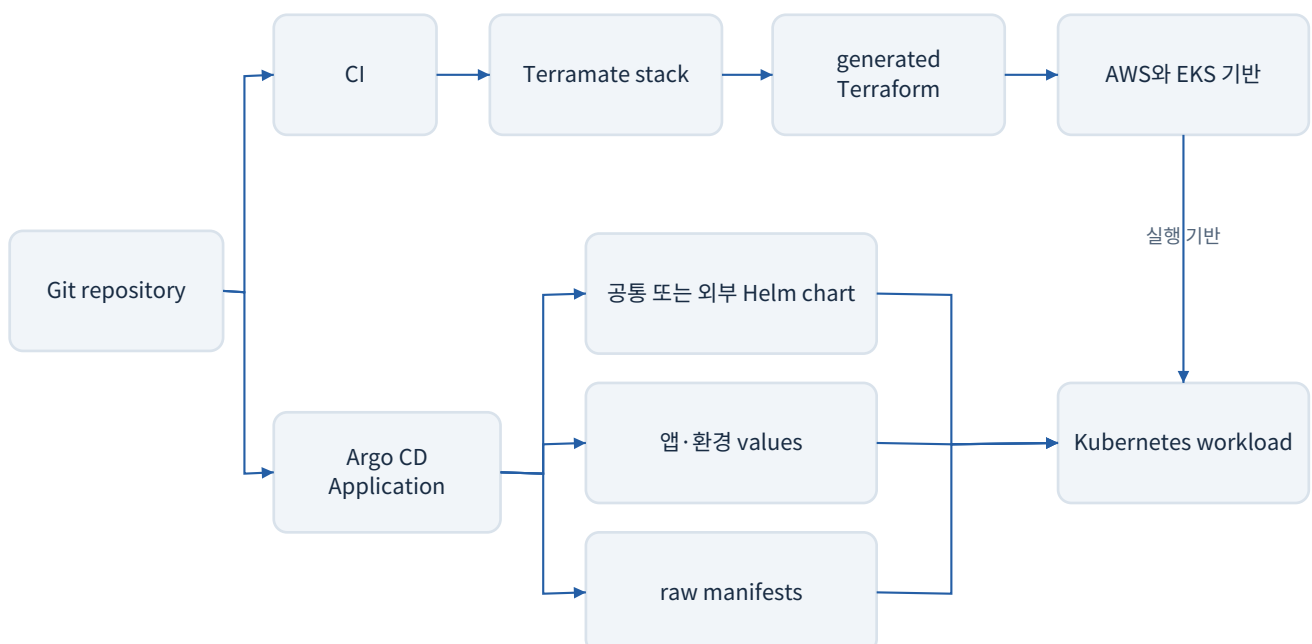
반대급부는 권한과 변경 변경이 한 저장소로 모인다는 점이다. 실행 범위는 --changed와 환경 tag가 stack 단위로 좁힌다. 권한은 아직 좁히지 않았다. 이 스냅샷의 CODEOWNERS는 단일 소유자 규칙 하나이고, 팀이 커지면 경로별 소유자로 나누는 것이 저장소를 다시 쪼개는 것보다 먼저 쓸 수단이다.

2. 저장소를 역할로 읽기

최상위 폴더는 도구 이름보다 변경 책임을 드러낸다. Terraform/Terramate, Argo CD, Helm values, raw manifest가 서로 다른 변경 경계를 갖고 한 저장소 안에서 연결된다.

```
devops-configs-portfolio/
├── _bootstrap/      # 원격 state와 CI 신뢰 기반을 먼저 만드는 스택
├── stacks/          # 환경별 Terraform 실행 단위
├── modules/         # Terramate generate_hcl 공통 모듈
├── imports/         # provider.backend 생성 규칙
├── charts/
│   ├── app/        # 일반 서비스용 공통 Helm 차트
│   └── batch/       # 별도 배치 차트(현재 주 경로는 아님)
├── values/
│   ├── apps/       # 애플리케이션·환경별 값
│   └── infra/       # 외부 인프라 차트의 값
├── argocd/          # Argo CD Application 선언
└── manifests/       # 차트 밖의 raw Kubernetes 리소스
```

이 구조에는 크게 두 흐름이 있다.



첫 번째 흐름은 Terraform이 클라우드·클러스터 기반과 일부 bootstrap 구성요소를 만든다. 두 번째 흐름은 Argo CD가 공통 차트, 외부 차트, raw manifest를 원하는 Kubernetes 리소스로 연결한다. 소스: [루트 구조](#), [Terramate stacks](#), [Argo CD 선언](#)

3. 스택 경로가 표현하는 설계 축

대표적인 스택 경로는 다음처럼 읽는다.

```
stacks/acme/eks/main/dev
  |   |   |   |
  |   |   |   |
domain kind target env

stacks/acme/ecr/mall/prd
  |   |   |   |
  |   |   |   |
domain kind group env
```

축	의미	EKS 예	ECR 예
domain	조직 또는 플랫폼 경계	acme	acme
resource kind	관리 대상 종류	eks	ecr
target / service group	대상 클러스터나 서비스 묶음	main	mall
environment	격리된 배포 환경	dev	prd

입사 시점 저장소에서는 환경이 디렉터리, 최상위 폴더, workspace 이름 중 어디에 있는지가 stack마다 달랐다(1-1). 현재는 환경이 항상 leaf 디렉터리이고, 루트 config.tm.hcl의 environment = terramate.stack.path.basename 한 줄이 그 규칙을 정의한다.

경로 깊이는 고정 스키마가 아니다. 예를 들어 outline/prd처럼 서비스 자체가 중간 축이 되거나, 리소스 종류에 따라 target 축이 생략될 수 있다. 따라서 “끝에서 두 번째는 항상 서비스” 같은 위치 기반 규칙보다 각 상위 config.tm.hcl이 정의하는 global을 함께 읽어야 한다.

EKS의 main은 버전 디렉터리가 아니다. stacks/acme/eks/main/config.tm.hcl에서 version = "v2"를 별도 global로 정의한다. dev의 Kubernetes 버전 1.35도 tfvars.tm.hcl에 따로 있다. 즉 main, v2, 1.35는 각각 논리적 대상, 클러스터 세대, Kubernetes 버전이라는 다른 관심사다.

ECR은 같은 계층 규칙 안에서도 구현 선택이 다르다. stacks/acme/ecr/mall/prd/main.tf는 리포지터리와 lifecycle policy를 직접 작성한다. 모든 leaf stack이 공통 모듈에서 생성된다는 전제는 맞지 않는다.

4. Terramate 상속과 생성 경계

Terraform Cloud와 환경 브랜치를 떠난 이유

도구를 고르기 전에 먼저 기존 실행 모델을 떠나야 할 이유가 셋 있었다.

문제	입사 시점의 모습	현재의 답
비용	Terraform Cloud가 유료 요금제 대상이 되어 실행·state 보관만으로 비용이 추가됨	S3 backend + GitHub Actions. 이미 쓰는 자원으로 대체

문제	입사 시점의 모습	현재의 답
환경 브랜치	dev·stage·prod 브랜치에 push하면 각각 트리거. 같은 변경을 세 브랜치에 맞춰 올려야 했고 배포 절차도 그만큼 길었음	main 하나와 feature 브랜치. PR에서 plan, main merge에서 apply
workspace	환경이 경로가 아니라 terraform.workspace 이름 안에 있어, 코드만 읽어서는 어느 환경에 적용되는지 알기 어려움(1-1)	환경은 항상 leaf 디렉터리. terraform.workspace 참조는 0

환경 브랜치의 핵심 문제는 **환경 차이가 브랜치 간 diff로 표현된다**는 점이다. 브랜치가 어긋나면 그것이 의도한 환경 차이인지 아직 반영하지 않은 변경인지 구분할 수 없다. 현재 구조는 환경 차이를 한 브랜치 안의 디렉터리와 values 파일로 옮겨서, 어긋남이 생길 자리를 없앤다.

환경 선택은 브랜치 대신 stack tag가 맡는다. deploy.yml은 main push 하나로 시작해 --tags=dev --changed와 --tags=prd --changed job을 각각 돌리고, preview.yml은 PR에서 같은 선택 규칙으로 plan을 보여준다.

Terragrunt 대신 Terramate를 선택한 이유

가장 중요한 선택 기준은 **단순성과 전달 가능성**이었다. 인프라 코드는 작성자 혼자 보는 코드가 아니므로, 새로 합류한 사람도 폴더를 따라가며 공통 규칙과 stack별 차이를 빠르게 이해할 수 있어야 했다. Terramate는 Terraform/HCL과 비슷한 표현을 유지하고 생성 결과도 native .tf로 남긴다. 별도의 큰 추상화 체계를 익혀야 하는 부담이 작고, 최종 Terraform 구성을 직접 확인할 수 있다는 점을 높게 평가했다.

단순한 구조를 유지하면서도 globals, import, generate_hcl로 반복을 제거할 수 있다. 공통 규칙은 상위 디렉터리에 두고 환경별 차이만 leaf stack에 남기므로, stack이 늘어나도 같은 구조를 복제해 확장하기 쉽다. ECR처럼 필요한 곳에는 직접 작성한 main.tf를 함께 둘 수 있어 모든 리소스를 하나의 생성 방식으로 강제하지도 않는다.

Terragrunt도 include, dependency, remote state 구성과 run queue를 제공하는 유효한 대안이다. 다만 이 저장소에서는 wrapper 설정을 중심으로 실행을 구성하는 방식보다, 읽을 수 있는 Terraform 파일을 생성하고 Git 변경 단위로 stack을 선택하는 Terramate의 모델이 팀의 학습과 유지보수에 더 단순하다고 판단했다. [Gruntwork 공식 문서](#)와 [Terragrunt include 문서](#)에서 Terragrunt의 모델을 확인할 수 있다.

비교 관점	Terramate를 선택한 이유	Terragrunt의 중심 모델
팀의 이해 비용	Terraform/HCL과 가까운 규칙과 생성된 .tf를 함께 확인	terragrunt.hcl의 include·dependency 규칙을 이해
DRY와 확장	상위 globals·import·generate_hcl을 상속하고 leaf에는 차이만 선언	include와 module source 입력을 재사용
기존 코드와 공존	생성된 .tf와 직접 작성한 Terraform을 같은 stack에서 사용	wrapper 설정이 Terraform/OpenTofu 실행을 둘러쌌
변경 실행	Git-aware --changed와 tags로 CI 대상 stack 선택	dependency graph 기반 run queue로 unit 실행 조정
운영 가시성	Terramate Cloud에서 stack 상태와 drift·배포 이력을 조회	필요에 맞는 별도 플랫폼 또는 연동을 함께 선택

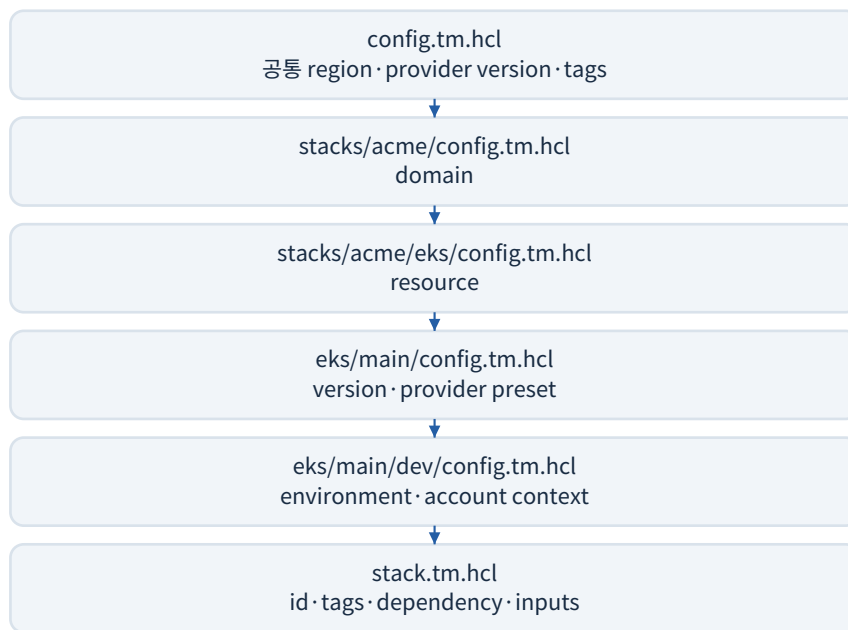
modules/eks/main.tm.hcl의 generate_hcl은 module 호출을 native .tf로 만들고, provider와 backend도 leaf에 생성한다. GitHub Actions의 deploy.yml은 --changed와 환경 tag를 함께 사용해 plan/apply 대상을 고른다. 이 구조는 Terramate의 [code generation](#)과 [change detection](#)을 각각 코드 공통화와 배포 범위 계산에 사용한다.

Terramate Cloud도 선택을 뒷받침했다. UI에서 등록된 stack 수와 상태를 한눈에 보고, stack별 drift 결과와 배포 이력을 확인할 수 있어 저장소가 커져도 운영 상태를 추적하기 쉽다. 현재 Community 플랜은 무료로 제공되지만, 2명·1,000개 리소스·30일 데이터 보존 범위라는 제한이 있다. 자세한 기능과 제한은 [Terramate Cloud stack 문서](#), [drift 문서](#), [공식 요금표](#)를 기준으로 한다.

선택 당시 지속적인 버전 업데이트와 빠른 제품 발전도 긍정적으로 평가했다. 이는 구조를 결정한 핵심 근거라기보다, 장기 운영에 사용할 도구라는 판단을 강화한 요소였다.

EKS dev stack은 after와 input으로 실행 순서와 VPC output 전달을 각각 선언한다. 다만 Terramate 공식 문서에서 [outputs sharing](#)은 experimental 기능으로 표시되며, 이 저장소의 workflow도 --enable-sharing을 명시한다. 이 기능은 도구 버전과 실행 기능 상태를 함께 관리해야 한다.

Terramate 설정은 루트에서 leaf 방향으로 맥락을 좁힌다.



구성 요소	책임
Terramate globals	디렉터리 계층에서 공유할 생성 시점 값과 규칙
import	provider preset과 resource module의 생성 규칙 선택
stack.tm.hcl	실행 단위의 ID, 태그, 순서, 외부 입력 선언
tfvars.tm.hcl	해당 환경의 Terraform 변수 파일 생성
Terraform locals	생성된 Terraform 안에서 값을 조합
Terraform variables	module 입력 계약과 기본값 정의

EKS target의 import.tm.hcl은 provider 생성 규칙과 /modules/eks/*.tm.hcl을 가져온다. ECR에는 imports.tm.hcl처럼 복수형 파일명도 있다. 파일명 자체보다 Terramate의 import 블록이 무엇을 선택하는지가 핵심이다.

공통 EKS 모듈은 generate_hcl로 Terraform 파일을 만든다. modules/eks/main.tm.hcl은 내부에서 terraform-aws-modules/eks/aws를 호출하고 허용할 버전 범위를 지정한다.

관련 필드 발췌:

```
generate_hcl "_terramate_generated_main.tf" {
  content {
    module "eks" {
      source = "terraform-aws-modules/eks/aws"
      version = "~> 21.9.0"
      # 입력은 locals와 variables에서 조합
    }
  }
}
```

`_terramate_generated_*.tf`와 `_generated_*.tf`는 생성 결과물이다. 소유권은 원본 `.tm.hcl`에 있으므로, 동작을 바꿀 때 생성 파일을 직접 고치는 대신 생성 규칙을 수정하고 다시 생성하는 구조다.

실행 순서와 값 공유는 별개다

EKS dev stack은 두 관계를 함께 표현한다.

관련 필드 발췌:

```
stack {
  after = ["tag:dev:vpc"]
}

input "vpc_id" {
  from_stack_id = "acme_vpc_main_dev"
  value         = outputs.vpc_id.value
}
```

`after`는 VPC 다음에 EKS를 실행하라는 순서 제약이다. `input`은 VPC stack output을 EKS input으로 전달하는 값 계약이다. 이 stack은 실행 순서와 값 전달을 각각 명시해 두 측면을 분리해서 표현한다.

입사 시점 저장소는 같은 값을 `terraform_remote_state`로 읽었다. 참조 키가 `"network-${local.remote_env[local.env]}"` 같은 workspace 이름 문자열이었고, 실행 순서는 코드에 선언되지 않았다. 현재는 참조 키가 stack ID이고 순서가 `after`로 남는다.

backend는 환경과 stack ID를 분리한다

생성된 S3 backend는 환경마다 다른 bucket을 사용하고, stack ID의 구성 요소로 key를 만든다.

관련 필드 발췌:

```
backend "s3" {
  bucket = "<environment-specific-state-bucket>"
  key     = "acme/eks/main/v2/dev/terraform.tfstate"
  profile = "dev"
}
```

입사 시점 저장소에서는 stack마다 backend "remote" 블록과 workspace 이름을 직접 썼다(1-1). 현재는 `imports/backend.tm.hcl` 하나가 `global.domain·global.environment`로 bucket을, `terramate.stack.id`로 key를 조합한다.

EKS key에 경로에 없는 v2가 포함되는 점이 중요하다. 이는 단순한 폴더 경로 복사가 아니라 stack identity로부터 backend 주소를 구성한다는 뜻이다. 실제 생성 결과는 `_terramate_generated_backend.tf`에서 확인할 수 있다.

5. 애플리케이션 공통 Helm 차트

charts/app는 여러 서비스가 Kubernetes workload 골격을 공유하기 위한 차트다.

공통화 대상은 templates와 helpers다. 각 앱의 values가 생성 여부와 필요한 설정을 제공하고 template이 생략 가능한 값의 fallback을 처리한다. 공통 차트가 필요한 이유는 여러 서비스에서 반복되던 Kubernetes template과 생성 규칙을 한곳에서 관리하기 위해서다.

```
공통 chart templates·helpers = 리소스를 만드는 규칙
서비스·환경별 values         = 규칙에 전달하는 값
```

서비스마다 리소스 구조가 크게 다르면 별도 차트가 더 적합하다. 반대로 Deployment·Service·HPA·PDB 등의 골격이 같고 이미지, 포트, probe, 리소스, 스케일링만 달라진다면 공통 차트가 서비스별 차트 복제를 줄인다. 현재 저장소의 charts/app은 후자의 범위를 공통화한다.

개편 전후의 mall v4 API 경로를 연결하면 공통화의 이동이 더 분명하다.

```
개편 전
acmemall-backend-v4/api/helm/{Chart.yaml, templates/*}
acmemall-backend-v4/api/values/dev.yaml
      ↓
현재
charts/app/{Chart.yaml, templates/*}      # 공통 골격
values/apps/mall/v4/api/dev.yaml          # 앱·환경 차이
argocd/dev/apps/mall/v4-api.yaml          # 둘을 연결
```

개편 전 API chart와 dev values는 서비스 디렉터리 안에 함께 있었다. 현재는 공통 app chart, mall v4 API dev values, Argo CD Application으로 책임이 나뉜다.

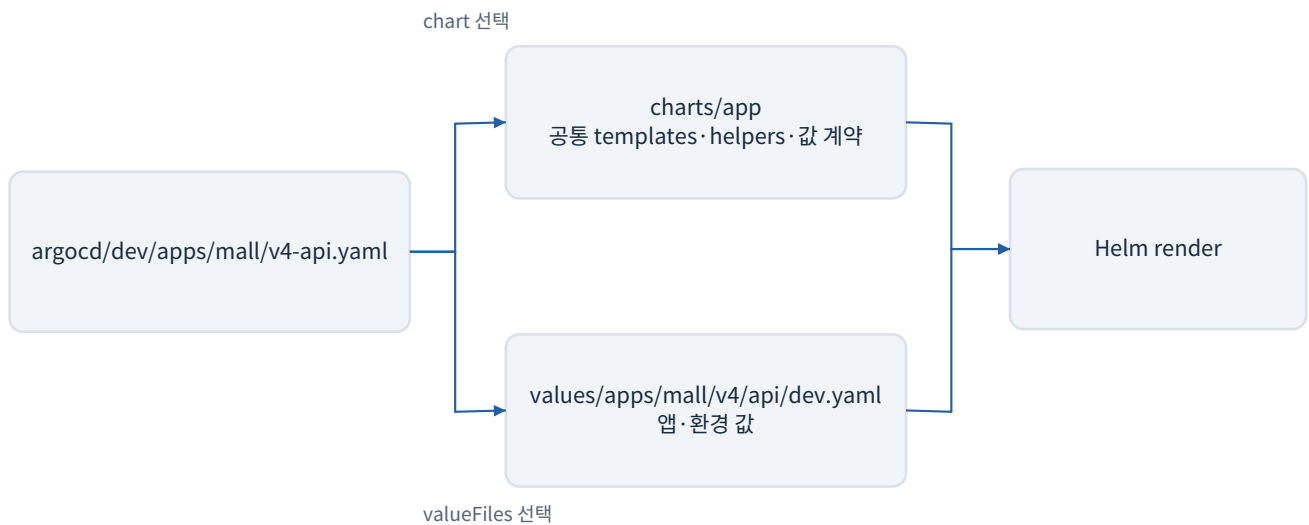
template	생성 리소스	활성 조건 또는 역할
deployment.yaml	Deployment	기본 workload
service.yaml	Service	service.enabled
serviceaccount.yaml	ServiceAccount	serviceAccount.create
hpa.yaml	HorizontalPodAutoscaler	autoscaling.enabled
pdb.yaml	PodDisruptionBudget	podDisruptionBudget.enabled
externalsecret.yaml	ExternalSecret	externalSecret.enabled

실제 template 목록에는 Namespace template이 없다. values의 namespace.name은 생성 리소스의 namespace를 지정하지만, 이것만으로 Namespace 객체를 생성한다고 해석하면 안 된다.

_helpers.tpl은 이름과 selector label, 이미지 주소, 환경 변수, 볼륨, node selector·affinity·toleration·topology spread 같은 반복 로직을 중앙화한다. 각 template은 이 helper를 호출해 동일한 규칙을 사용한다.

공통 차트와 앱·환경 values의 결합

공통화의 핵심은 charts/app의 templates와 helpers가 제공하는 리소스 골격 및 값 계약이다. Argo CD Application은 이 공통 차트와 앱·환경별 values 파일을 선택해 Helm에 전달한다.



mall-v4-api dev Application은 다음 연결을 명시한다.

관련 필드 발췌:

```

source:
  path: charts/app
  helm:
    valueFiles:
      - ../../values/apps/mall/v4/api/dev.yaml
  
```

values 경로의 축은 apps / domain / generation / component / environment로 읽을 수 있다. 이 사례는 mall / v4 / api / dev다. 폴더 이름만으로 중간 values가 자동 상속되지는 않는다. Application이 명시한 앱·환경 파일이 서비스 설정의 입력이다.

실제 dev values의 관련 필드 발췌:

```

image:
  repository: acmemall-backend-v4/dev/api
  tag: latest

service:
  enabled: true
  port: 80
  targetPort: 8080

resources:
  requests:
    cpu: 35m
    memory: 1200Mi

autoscaling:
  enabled: false
  
```

deployment.yaml은 image와 resources를 컨테이너 필드에 넣고, service.yaml은 port 80을 컨테이너의 target port 8080에 연결한다. autoscaling이 꺼진 dev에서는 HPA를 만들지 않고 Deployment가 replica 수를 가진다.

dev와 prd가 같은 골격을 다르게 사용하는 방식

dev values와 prd values는 같은 chart를 사용하지만 운영 파라미터가 다르다.

항목	dev	prd	template 효과
autoscaling	enabled: false	enabled: true	dev는 Deployment replicas 사용, prd는 HPA 생성
replica 범위	Deployment 1	HPA min 2, max 4	prd의 replica 결정권을 HPA가 가짐
Service	port 80 → target 8080	port 80 → target 8080	같은 서비스 계약 유지
requests	CPU 35m, memory 1200Mi	CPU 84m, memory 1500Mi	환경별 workload 입력
memory limit	1200Mi	2000Mi	환경별 workload 입력
datadog.enabled	false	true	helper가 관련 label과 env를 구성
PDB	기본 비활성	minAvailable: 1	prd에 PDB 생성
scheduling	topology spread 비활성	명시적 분산 규칙	Pod 배치 제약 변경

deployment.yaml은 autoscaling이 켜지면 spec.replicas를 아예 출력하지 않는다. hpa.yaml이 min/max와 metric을 관리하고, Application은 Deployment의 /spec/replicas 차이를 무시한다. 이 세 설정이 함께 replica 소유권 충돌을 줄인다.

표의 PDB와 scheduling 행이 왜 그렇게 설정되었는지는 [EKS 워크로드 분산 설계](#)에서 따로 다룬다.

고정 스냅샷에서 `helm template mall-v4-api charts/app --namespace mall -f values/apps/mall/v4/api/dev.yaml`과 `prd` 대응 명령으로 로컬 렌더링을 확인했다. dev는 ServiceAccount·Service·Deployment를, prd는 여기에 PDB와 HPA를 추가로 생성했다. 이는 template 출력 검증이며 클러스터 health 검증은 아니다.

charts/batch도 존재하지만, 이 스냅샷에서 v4-batch Application은 주석 처리되어 있고 실제 v4-batch-server는 charts/app을 사용한다. 따라서 batch 차트를 현재 공통 배포 경로의 중심으로 보기는 어렵다. 또한 batch의 commonValues와 app은 값 그룹이며, 임의의 deep merge 계층으로 해석할 근거는 없다.

6. 공통 앱 차트 바깥의 배포 경로

모든 Helm 배포가 charts/app이나 Argo CD 하나로 수렴하지는 않는다.

경로	예시	책임
외부 Helm chart + Argo CD	Traefik	chart 버전과 환경 values 연결
raw manifests + Argo CD	route, priority class 등	차트화 이점이 적은 리소스 선언
Terraform Helm provider	Argo CD, Karpenter 등 bootstrap	GitOps 제어면을 시작하기 위한 설치
공통 앱 chart + Argo CD	mall API	서비스 workload 표준화

Traefik dev Application은 외부 chart 38.0.0과 저장소의 values/infra/traefik/dev.yaml을 multi-source로 묶고 CreateNamespace=true를 지정한다.

관련 필드 발췌:

```
sources:
- ref: values
- chart: traefik
  targetRevision: 38.0.0
  helm:
    valueFiles:
    - $values/values/infra/traefik/dev.yaml
```

여기서 외부 chart는 upstream 리소스 모델을 제공하고, values/infra는 환경 정책을 담는다. 애플리케이션 workload의 values/apps와 인프라 addon의 values/infra를 나눠 변경 책임을 드러낸다. Terraform과 Argo CD의 Helm 책임도 구분해야 한다. Terraform 쪽 Helm stack은 Argo CD 자체나 클러스터 핵심 구성요소를 설치해 GitOps 제어면이 작동할 기반을 만든다. 그 뒤 Argo CD Application은 일반 앱과 많은 인프라 addon을 지속적으로 동기화한다. “모든 Helm은 Argo CD만 사용한다”는 설명은 이 bootstrap 경계를 놓친다.

7. 변경 경계와 트레이드오프

공통화는 중복을 줄이는 대신 변경 환경을 만든다. 이 저장소는 값, template, module, stack을 분리해 그 변경을 파일 위치로 표현한다.

변경 위치	적합한 변경	예상 영향 범위	검토 초점
values/apps/.../ <env>.yaml	한 앱·한 환경의 리소스, probe, scaling	해당 Application	환경별 의도와 chart 계약
values/infra/.../ <env>.yaml	외부 chart의 환경 정책	해당 인프라 addon	upstream chart 버전과 값 호환성
charts/app/templates	모든 공통 앱의 리소스 구조	차트 소비 앱 전체	기존 values와 조건부 렌더링
modules/<resource>/*.tm.hcl	여러 stack의 Terraform 생성 규칙	해당 module import stack	생성 결과와 provider/module 계약
stacks/.../<env>	특정 대상·환경의 입력과 직접 리소스	해당 leaf stack과 output 소비 stack	backend, 의존성, 환경 격리

이 설계의 장점은 공통 골격과 환경 차이를 분리하면서도 최종 연결을 코드로 추적할 수 있다는 점이다. 반대급부는 공통 chart나 module의 작은 변경도 여러 소비자에게 전파될 수 있다는 점이다. 따라서 변경 위치를 선택할 때 먼저 질문할 것은 “이 동작이 한 앱·한 환경의 예외인가, 여러 소비자가 공유할 계약인가”다. 전자는 values나 leaf stack에, 후자는 공통 chart나 Terramate module에 두는 것이 저장소의 현재 경계와 맞는다.

노드 한 대가 빠져도 서비스가 남도록: EKS 워크로드 분산 설계

운영 EKS 클러스터에서 노드 한 대가 NotReady가 되자 서비스가 같이 흔들린 장애를 계기로, 워크로드 분산을 배치·재배치·보호·노드 공급 네 층으로 다시 설계한 내용을 설명한다.

근거	대상	기준
설정	devops-configs-portfolio	commit 7e3427a 정적 스냅샷
재현	이 저장소의 kind 클러스터 (Kubernetes v1.35, 운영과 같은 minor)	scripts/topology-spread-repro.sh

스냅샷은 설정을, 재현은 스케줄러 동작을 보여준다. 운영 클러스터의 현재 상태를 증명하는 문서는 아니다. 저장소 구조 자체는 [GitOps 저장소 아키텍처](#)와 [공통화 설계](#)에서 다룬다.

1. 문제: 노드 한 대가 곧 서비스 한 개였다

운영 서비스에서 간헐적인 장애가 이어졌다. 추적해 보니 pod들이 노드 한쪽에 몰려 있었고, 그 노드가 메모리 포화로 NotReady가 되면 그 위의 서비스가 함께 내려갔다. replica는 여러 개였지만 노드 기준으로는 하나였던 셈이다.

쏠림이 만드는 위험은 두 단계다.

빠지는 것	쏠려 있을 때
노드 한 대	그 노드에 몰린 앱은 replica가 전부 사라짐
AZ 하나	그 AZ의 노드에 몰린 앱 전체가 같은 일을 겪음. 노드를 여러 AZ에 둔 의미가 없어짐

몰린 이유는 분산 제약이 없거나 권고였기 때문이다.

상태	결과
topologySpreadConstraints 없음	기본 분산 선호와 다른 스케줄링 조건에 따라 배치되며, 앱의 강제 분산은 보장하지 않음
whenUnsatisfiable: ScheduleAnyway	maxSkew: 1이 있어도 점수에만 반영. 압박을 받으면 한 노드에 전부 들어감
재배치 수단 없음	노드 재기동 뒤 한 번 몰리면 그대로 고착

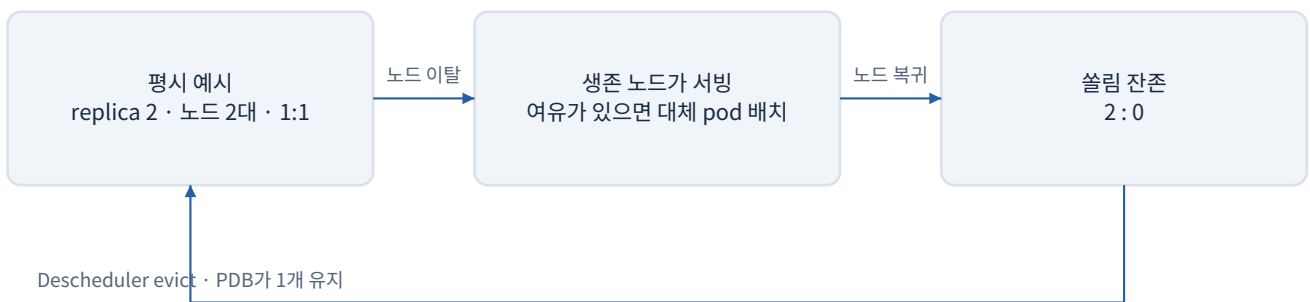
공통 차트의 기본값이 바로 그 권고였다. `_helpers.tpl`의 기본 제약은 `zone`과 `hostname` 모두 `ScheduleAnyway`로 렌더링된다.

이 문제가 특히 아팠던 배경이 있다. 노드 수를 줄이려고 대형 인스턴스를 AZ당 1대씩 두는 구성이라, 노드 한 대가 곧 용량의 절반이다. 노드가 적을수록 분산은 선택이 아니라 전제가 된다.

2. 설계: 네 개의 층

한 도구로는 풀리지 않는다. 각 도구가 **못 하는 일**을 다음 층이 받는 구조다.

층	도구	맡는 일	못 하는 일
배치	TopologySpreadConstraints	스케줄 시점에 쏠림을 차단	이미 놓인 pod은 옮기지 않음
재배치	Descheduler	복구 뒤 남은 쏠림을 evict로 교정	어디로 갈지는 정하지 않음 (스케줄러 몫)
보호	PodDisruptionBudget	자발적 eviction에서 최소 replica 보장	노드 장애 같은 비자발적 중단은 못 막음
노드 공급	Karpenter NodePool	AZ별 공급 경로와 NodePool별 중단 예산 설정	기존 pod을 직접 재배포하지 않음



3. 배치: TopologySpreadConstraints

운영 앱의 제약은 두 줄이다. mall v4 API prd values의 관련 필드 발췌:

```

topologySpreadConstraints:
  - topologyKey: topology.kubernetes.io/zone
    maxSkew: 1
    whenUnsatisfiable: ScheduleAnyway # zone 은 best-effort
    nodeTaintsPolicy: Honor
    matchLabelKeys: [pod-template-hash]
  - topologyKey: kubernetes.io/hostname
    maxSkew: 1
    whenUnsatisfiable: DoNotSchedule # 적격 hostname 간 개수 차이를 1 이하로 제한
    nodeTaintsPolicy: Honor
    matchLabelKeys: [pod-template-hash]
  
```

maxSkew: 1은 노드당 1개 제한이 아니다. 적격 노드 2대에서 동일 리비전의 replica가 2개면 1:1, 3개면 2:1도 허용한다. 명시적 TSC가 없어도 스케줄러의 기본 분산 선호는 적용될 수 있다. [Kubernetes TSC 정의와 기본 동작](#)

hostname만 강제하고 zone은 권고로 둔 이유는 AZ당 적격 앱 노드 1대를 운영하는 구성에 있다(6절). 이 조건에서는 hostname 분산이 AZ 분산으로 이어진다. zone은 장애 중 배치 유연성을 우선해 권고로 두었으며, 노드 수나 AZ별 분포가 달라지면 zone 제약도 재검토해야 한다.

필드	없으면 생기는 일
whenUnsatisfiable: DoNotSchedule	제약이 권고로 남아 실행을 허용
nodeTaintsPolicy: Honor	갈 수 없는 노드가 분산 계산에 끼어 Pending (3-1, 3-2)
matchLabelKeys: [pod-template-hash]	구·신 버전을 합산해 신버전 분포가 치우치거나 롤아웃이 지연될 수 있음 (3-3)

세 필드를 함께 검토했다. 강제 분산만 켜면 taint와 롤아웃 중 리비전 합산으로 생기는 배치 문제를 놓칠 수 있다.

3-1. Honor가 없으면: 유령 도메인

기본값 ignore는 **pod이 갈 수 없는 taint 노드도 분산 도메인으로 센다**. 이 클러스터에는 batch·airflow 전용 NodePool이 taint를 달고 있다(batch.yaml). 앱 pod 입장에서 그 노드들은 영원히 0개인 도메인이다.

replica 3, 앱 노드 2대 + taint 노드 1대				
Ignore	app-1: 1	app-2: 1	tainted: 0	3번째를 app-1 에 놓으면 2 vs 0 = skew 2 → Pending
Honor	app-1: 1	app-2: 1		3번째를 app-1 에 놓으면 2 vs 1 = skew 1 → 배치

replica 2는 1:1에서 멈추므로 우연히 안전하고, replica 3부터 터진다. 그래서 테스트에서 놓치기 쉽다.

로컬 재현(taint 걸린 infra 노드 1대 + app 노드 2대):

설정	결과
Ignore (기본값)	2개 Running, 1개 Pending - 2 node(s) didn't match pod topology spread constraints, 2 node(s) had intolerated taint(s)
Honor	3개 Running, 2:1

3-2. Honor의 두 번째 효과: 노드가 빠진 동안의 복구

노드가 NotReady가 되면 노드 컨트롤러가 `node.kubernetes.io/not-ready:NoSchedule` taint를 붙인다. 스케줄러에게 그 노드는 3-1의 taint 노드와 같은 존재가 된다. 즉 Ignore에서는 **죽은 노드가 0개짜리 도메인으로 남아, 살아 있는 노드에 대체 pod을 올리는 것을 막는다**.

로컬 재현(replica 2, 한 노드에 tolerate하지 않는 NoSchedule taint를 걸고 그 노드의 pod을 삭제):

설정	노드가 빠진 동안	의미
Ignore	대체 pod Pending	장애 내내 replica 1개로 버팀
Honor	대체 pod이 생존 노드에 Running	생존 노드에 여유가 있으면 스케줄 후 Ready 완료 시 서빙 용량 회복 가능

강제 분산은 “노드 장애에 대비한다”는 목적과 반대로, 장애 중 복구를 막을 수 있다. Honor가 그 모순을 푼다. 재현은 실제 노드 정지가 아니라 스케줄러가 보는 조건(tolerate하지 않는 taint)을 만든 것이다.

3-3. matchLabelKeys: 리비전별 분산

롤링 업데이트 중에는 구버전과 신버전 pod이 같은 selector에 함께 잡힌다. 이 합산이 항상 배포를 막는 것은 아니지만, 구버전 삭제 순서와 노드 여유에 따라 신버전 분포가 치우치거나 배치가 지연될 수 있다. pod-template-hash를 키로 주면 리비전별로 따로 쏜다. [Kubernetes matchLabelKeys](#)

3-4. 적용 방식: 차트 기본값이 아니라 앱별 values

차트 helper를 고치면 한 줄로 끝나지만, 그 차트를 쓰는 모든 앱이 동시에 돌아옴된다. JVM 앱은 기동 시 CPU와 메모리를 크게 쓰므로, 분산을 고치려다 노드 압박을 다시 만들 수 있었다.

방식	돌아옴 범위	선택
공통 차트 helper 수정	차트 소비 앱 전체가 동시에	보류
앱별 values에 명시 (defaultTopologySpread.enabled: false)	그 앱만	채택. non-JVM 먼저, JVM은 2~3개씩 나눠 적용하고 매번 분포 확인

아키텍처 문서 7절의 질문 - 한 앱의 예외인가, 공유 계약인가 - 을 **변경 시점의 위험** 기준으로 한 번 더 적용한 사례다. 최종적으로는 공유 계약이 되어야 하지만, 거기까지 가는 길은 앱 단위로 끊었다.

4. 재배치: Descheduler

TSC는 스케줄 시점에만 작동한다. 3-2의 재현을 이어가면 그 한계가 보인다.

시점	Honor 쪽 분포
노드가 빠진 동안	생존 노드에 2
노드 복구 뒤	여전히 2 : 0 - 아무도 옮기지 않음

용량을 지킨 대가로 쏠림이 남는다. 처음 장애의 원인과 같은 상태다. 이것을 주기적으로 교정하는 것이 Descheduler다. `descheduler prd values`의 관련 필드 발췌:

```
kind: CronJob
schedule: "*/10 * * * *"
deschedulerPolicy:
  profiles:
    - name: topology-spread-rebalance
      pluginConfig:
        - name: RemovePodsViolatingTopologySpreadConstraint
          args:
            constraints: [DoNotSchedule]
            namespaces:
              include: [mall, healthcare, default]
      plugins:
        balance:
          enabled: [RemovePodsViolatingTopologySpreadConstraint]
```

범위를 의도적으로 좁혔다.

결정	이유
플러그인은 RemovePodsViolating TopologySpreadConstraint 하나	LowNodeUtilization 같은 넓은 전략은 evict 대상이 예측하기 어려움. 풀려는 문제는 분산 위반 하나
constraints: [DoNotSchedule]만	강제된 제약의 위반만 교정. 권고(zone) 위반으로는 evict하지 않음
namespace 한정	강제 분산을 적용한 앱 namespace만
10분 주기 CronJob	차트 기본값(2분)보다 느슨하게. 첫 배포는 보수적으로

배포는 Argo CD Application이 upstream chart와 저장소의 values를 multi-source로 묶는다.

5. 보호: PodDisruptionBudget

Descheduler를 넣는 순간 새 위험이 생긴다. 분산을 맞추려고 evict하다가 replica 2~3개짜리 서비스를 **한꺼번에** 내릴 수 있다. Descheduler는 Eviction API로만 pod를 내리고, Eviction API는 PDB를 확인한다. 그래서 Descheduler 쪽에 별도의 최소 replica 옵션을 두지 않고 PDB 하나로 막는다.

```
podDisruptionBudget:
  enabled: true
  minAvailable: 1
```

pdb.yaml template은 기본 `enabled: false`다. 차트에 기능을 추가해도 기존 릴리스의 렌더링 결과는 바뀌지 않고, 서비스별로 켜다.

같은 PDB가 Karpenter의 노드 교체(만료·consolidation)와 수동 drain에서도 작동한다. 반대로 노드 장애처럼 **비자발적** 중단에는 관여하지 못한다. 그쪽은 3절의 몫이다.

6. 노드 공급: Karpenter

pod을 노드에 나눠 놓으려면 노드가 먼저 나뉘어 있어야 한다. Karpenter에 맡겨 두면 두 노드가 같은 AZ에 뜰 수 있다.

설정	효과
NodePool을 AZ별로 분리 (base, base-b), 각각 zone requirement 고정	앱 노드가 서로 다른 AZ에 놓임
limits.cpu = 인스턴스 1대분	8vCPU 타입과 한도 8로 NodePool당 통상 1대 운영을 목표로 함
expireAfter를 서로 다르게 (720h / 1080h)	최초 만료 시점 분산. 이후 생성·교체 시점에 따라 겹칠 수 있음
disruption.budgets: nodes: "1"	각 NodePool의 drift·consolidation 등 예산 대상 중단을 최대 1대로 제한

CPU 한도 검사는 eventual consistency이므로 일시 초과할 수 있고, 각 AZ에 최소 1대 존재하는 것도 보장하지 않는다.

Karpenter limits

중단 예산은 NodePool별이므로 두 pool의 동시 중단을 막는 전역 제한이 아니다. 만료는 이 예산의 제어 대상이 아니며, 서로 다른 expireAfter도 동시 교체를 보장해 막지는 않는다. [Karpenter 중단 예산과 만료](#)

nodepool.yaml.tpl의 관련 필드 발췌:

```
spec:
  template:
    spec:
      expireAfter: ${expire_after}
      requirements:
        - key: topology.kubernetes.io/zone
          operator: In
          values: ["${availability_zone}"]
      limits:
        cpu: ${cpu_limit}
      disruption:
        consolidationPolicy: WhenEmptyOrUnderutilized
        consolidateAfter: 10m
      budgets:
        - nodes: "1"
```

첫 번째 NodePool은 **공통 모듈**이 생성하고, 두 번째는 **prd stack**에만 직접 작성되어 있다. dev에는 AZ 이중화가 필요 없기 때문이다. 공통 생성과 leaf 직접 작성을 섞는 방식은 아키텍처 문서 4절의 구조를 그대로 쓴다.

7. 장애 시나리오로 다시 읽기

단계	일어나는 일	작동하는 층
평시 예시	동일 리비전 replica 2개·적격 노드 2대에서 1:1	TSC hostname DoNotSchedule
노드 A NotReady	노드 B의 replica가 계속 서빙	위 배치의 결과
대체 pod 생성	죽은 노드를 계산에서 빼고, 여유가 있는 노드 B에 배치	TSC nodeTaintsPolicy: Honor
노드 복귀 또는 교체	분포는 2:0 그대로	(아무도 옮기지 않음)
10분 주기 교정 시도	PDB가 허용하면 위반 pod evict, 최소 1개 유지	Descheduler + PDB
재스케줄	비어 있는 노드로 배치되어 1:1	TSC

10분은 실행 주기이며 복구 완료 보장이 아니다. Job 실행 지연, PDB, 가용 용량과 pod Ready 시간에 따라 교정이 늦어질 수 있다. [Kubernetes CronJob](#) 한계

운영 환경에서도 단일 노드 이탈 시 워크로드가 다른 노드로 재배치되고 서비스가 유지되는 것을 확인했다. 이 문장은 운영 관찰이며 스냅샷으로 증명되는 내용은 아니다.

8. 검증한 것

대상	방법	결과
차트 렌더링	helm template mall-v4-api charts/app -n mall -f values/apps/mall/v4/api/prd.yaml	Deployment에 3절의 제약 두 줄, PDB minAvailable: 1, HPA 생성
차트 기본값	helm template t charts/app	zone·hostname 모두 ScheduleAnyway
유령 도메인 (3-1)	kind, replica 3	Ignore 1개 Pending / Honor 2:1
장애 중 복구 (3-2)	kind, replica 2, 노드 taint + pod 삭제	Ignore Pending / Honor 생존 노드에 Running
복귀 뒤 쏠림 (4절)	위 실험에서 taint 제거	Honor 쪽 2:0 유지

재현 스크립트는 임시 namespace를 만들고 끝나면 taint와 namespace를 지운다. Descheduler의 evict 동작은 로컬에서 재현하지 않았다.

9. 한계와 남은 과제

항목	상태
차트 기본값	여전히 ScheduleAnyway. values.yaml의 defaultTopologySpread.whenUnsatisfiable 키는 helper가 읽지 않아 값을 바꿔도 렌더링이 변하지 않는다. 앱별 적용이 끝났으므로 helper를 이 키에 연결해 기본값을 승격하는 것이 다음 단계
적용 범위	스냅샷 기준 prd 앱 values 23개 중 hostname 강제 10개, PDB 9개. 강제 분산을 쓰면서 Honor와 PDB가 빠진 앱이 1개 있다
생존 노드의 여유	Honor는 장애 중 pod을 생존 노드에 모은다. 그 노드가 전체 부하를 받을 requests 여유가 없으면 처음 장애(메모리 포화)를 옮겨 놓는 셈이다. requests 산정이 이 설계의 전제
대상 밖	PV가 AZ에 묶인 StatefulSet, 단일 replica, DaemonSet, EKS managed addon

계정 하나로 클러스터 도구와 AWS까지: Authentik SSO 설계

Kubernetes 위의 운영 도구와 AWS 접근을 하나의 IdP로 묶고, 그 IdP의 설정까지 Git으로 관리하게 만든 과정을 설명한다. 도구마다 지원하는 인증 방식이 달라서, 핵심은 "SSO를 붙였다"가 아니라 붙이는 방식을 앱의 능력에 맞춰 고른 것과 IdP 설정을 코드로 둔 것이다.

근거	대상	기준
설정	devops-configs-portfolio	commit 7e3427a 정적 스냅샷

스냅샷의 설정을 읽은 문서이며, 운영 클러스터의 현재 상태를 증명하지는 않는다. 저장소 구조는 [GitOps 저장소 아키텍처](#)와 [공통화 설계](#)에서 다룬다.

1. 문제: 도구 수만큼 계정이 있었다

증상	결과
Argo CD, Grafana, Airflow, Argo Workflows가 각자 계정을 가짐	입사자 한 명에 계정 생성 여러 번, 퇴사자 한 명에 삭제 여러 번. 하나라도 빠뜨리면 남은 계정이 생김
인증 기능이 아예 없는 도구가 있음	IP 제한에만 의존
AWS 콘솔은 IAM User, CLI는 access key	만료되지 않는 자격 증명이 사람 수만큼 존재

목표는 둘이었다. 회사 Google 계정 하나로 전부 로그인할 것, 그리고 장기 자격 증명을 없앨 것.

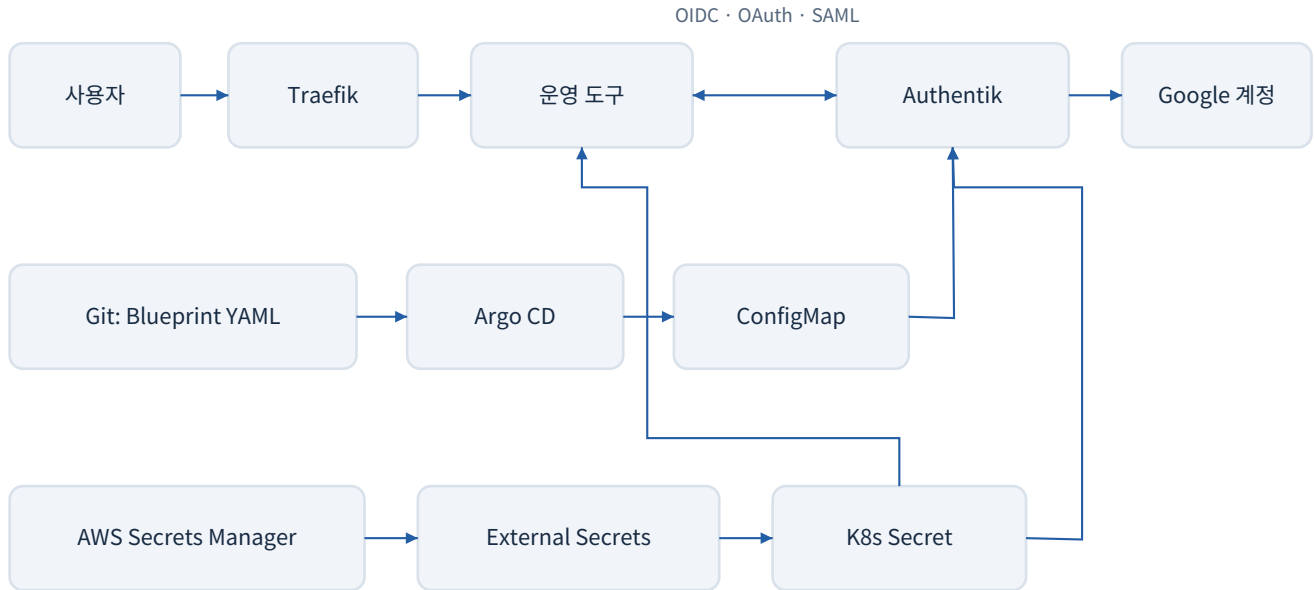
2. IdP 선택: 설정을 Git에 둘 수 있는가

관점	검토한 사용자당 과금 SaaS IdP	Keycloak	Authentik
형태	SaaS	self-hosted	self-hosted
과금	사용자당	오픈소스 자체 운영	오픈소스 자체 운영
설정 방식	제품별 API·자동화 지원 확인 필요	JSON import/export·Admin API	Blueprint (YAML)

결정적인 기준은 팀이 이해하기 쉬운 선언 방식이었다. [Keycloak도 설정 자동화가 가능](#)하지만, 애플리케이션·provider·정책·그룹을 YAML로 선언하는 Authentik Blueprint가 이 저장소의 GitOps 운영 방식에 가장 직접적으로 맞았다. 인증 설정도 코드 리뷰와 변경 이력에 포함할 수 있었다.

사용자당 과금은 팀이 커질수록 비용이 따라 커지는 구조라 먼저 제외했다.

3. 구조: 세 개의 경로



경로	흐르는 것	저장소 위치
로그인	사용자의 인증 요청	manifests/traefik/prd/infra.yaml
설정	앱·provider·정책·그룹 선언	manifests/authentik/prd/blueprints.yaml
시크릿	OAuth client secret	manifests/authentik/prd/external-secret.yaml

설정은 Git에 있지만 시크릿은 Git에 없다. 이 둘을 잇는 방법이 4절이다.

4. IdP 설정을 코드로: Blueprint

Blueprint들은 ConfigMap 하나에 담겨 Authentik pod에 마운트된다. authentik prd values가 그 ConfigMap을 지정하고, Argo CD Application이 동기화한다.

스냅샷 기준 규모:

항목	수
Blueprint 인스턴스	19
Application	20
OAuth2 provider / SAML provider	8 / 2
정책 바인딩	15

시크릿을 Git 밖에 두는 방법

Blueprint에는 client secret 자리에 환경 변수 참조만 적는다.

```
# blueprints.yaml - Git 에 있는 것
- model: authentik_providers_oauth2.oauth2provider
  attrs:
    client_id: grafana
    client_secret: !Env AUTHENTIK_GRAFANA_OAUTH_SECRET
```

```
# external-secret.yaml - 값은 AWS Secrets Manager 에서
- secretKey: AUTHENTIK_GRAFANA_OAUTH_SECRET
  remoteRef: { key: <cluster-secret>, property: <property> }
```

ExternalSecret이 만든 K8s Secret을 Authentik pod이 envFrom으로 받고, Blueprint의 !Env가 그 값을 읽는다. 앱 쪽 ExternalSecret도 Secrets Manager의 **같은 property**를 읽어 수동 복사에 따른 불일치를 줄였다. 갱신·pod 재시작 시점이 다르면 일시적으로 값이 다를 수 있다. 스냅샷에서 Blueprint의 !Env 변수 7개가 전부 ExternalSecret 키에 대응하는 것을 확인했다.

접근 모델도 선언이다

단계	선언된 동작
Google 계정으로 최초 로그인	제한 그룹에 자동 배정. 사내 위키만 보임
개발자로 승격	관리자가 개발 그룹에 추가 (수동, 한 번)
개발 도구 접근	앱마다 "개발 그룹만 허용" expression policy를 바인딩
퇴사	Authentik 계정 비활성화로 신규 인증 차단을 중앙화

신규 사용자의 기본 권한은 위키로 제한하고, 개발 도구 권한은 별도로 부여한다. 기존 앱 세션의 종료는 앱의 [Single Logout 지원·설정](#)에 의존하며, 이미 발급한 AWS 임시 자격 증명은 [만료·회수 정책](#)을 함께 확인해야 한다.

5. 앱 연동: 앱이 지원하는 만큼만

연동 방식은 선호가 아니라 **앱이 무엇을 지원하는가**로 정해진다.

앱의 능력	방식	대상
OIDC 내장	OIDC	Argo CD, Argo Workflows, 위키
OAuth는 되지만 discovery 미사용	Generic OAuth - auth_url·token_url·api_url을 직접 지정	Grafana (values)
프레임워크 수준의 OAuth	Flask OAuth	Airflow
인증 기능 없음	앞단에 oauth2-proxy를 reverse proxy로 배치	비용 분석 도구
SAML SP	SAML	AWS 콘솔

앱의 능력	방식	대상
CLI	OIDC Authorization Code + PKCE	AWS CLI (6절)

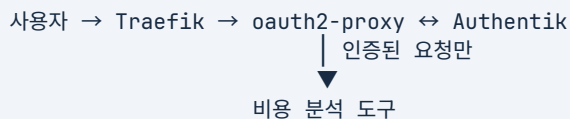
5-1. Argo CD: public client 하나로 웹과 CLI를 통합

이 구성에서는 **application마다 issuer가 다르다**. CLI용 provider를 따로 만들면 issuer가 /application/o/argocd-cli/가 되어, 서버가 기대하는 /application/o/argocd/와 어긋나 토큰 검증이 실패한다.

시도	결과
웹은 confidential, CLI는 별도 public provider	issuer 불일치로 CLI 로그인 실패
provider 하나를 public으로 두고 웹과 CLI가 공유	동작. CLI용 http://localhost:*/auth/callback redirect URI만 추가

CLI는 secret을 안전하게 보관하기 어려워 public client로 구성하고, authorization code 탈취를 완화하기 위해 **PKCE**를 사용했다. 로그인 시작 시 만든 일회성 값으로 코드 교환을 보호하는 방식이며, client secret을 통한 클라이언트 인증과는 역할이 다르다.

5-2. 인증이 없는 도구: oauth2-proxy



IngressRoute는 해당 호스트의 모든 트래픽을 proxy로 보낸다. 도구 자체에는 인증 설정이 없다.

이 구성에서는 cookie secret을 args로 전달하던 중 재배포 뒤 로그인 루프를 겪었다. OAUTH2_PROXY_COOKIE_SECRET 같은 네이티브 환경 변수로 전달 방식을 통일해 해결했다. 이 관찰만으로 Kubernetes의 \$(VAR) 치환 기능 자체가 불안정하다고 일반화하지는 않는다.

5-3. Argo Workflows: 무한 리다이렉트

Argo Workflows는 미인증 사용자를 스스로 SSO로 보내지 않는다. 그래서 Traefik middleware로 / 접근을 SSO로 리다이렉트했는데, 로그인을 마치고 돌아온 요청도 다시 /라서 끝없이 돌아다.

```
/ → SSO → 로그인 완료 → / → SSO → ...
```

Traefik은 로그인이 끝났는지 알 방법이 없다. 하지만 로그인 뒤에는 authorization 쿠키가 생긴다. 그래서 **쿠키 유무로 라우트**를 나눴다.

```

routes:
- match: Host(`...`) && HeaderRegexp(`Cookie`, `authorization=`) # 쿠키 있음 → 앱에서 검증
  priority: 20
- match: Host(`...`) && Path(`/`) # 미인증의 첫 진입 → SSO
  priority: 15
  middlewares: [argo-sso-redirect]
- match: Host(`...`) # 콜백·정적 파일 → 그대로 전달
  priority: 10
  
```

쿠키 유무는 리다이렉트 경로를 고르는 힌트이며 인증 판정이 아니다. [HttpOnly](#)는 브라우저 JavaScript의 쿠키 접근을 제한할 뿐 요청의 진위를 보장하지 않는다. 실제 세션·권한 검증은 Argo Workflows가 담당한다.

이 문제를 오래 끈 원인은 따로 있었다. Traefik v3에서 matcher 이름이 [HeadersRegexp](#)에서 [HeaderRegexp](#)로 바뀌었다. 당시 확인한 로그에서는 원인을 바로 찾지 못해 설계를 의심했지만, v3 문법으로 수정해 해결했다.

5-4. IdP 자체의 라우팅: 호출자가 누구인가

IdP 호스트에 IP 제한을 통째로 걸면 OIDC가 깨진다. 같은 호스트를 서로 다른 세 주체가 호출하기 때문이다.

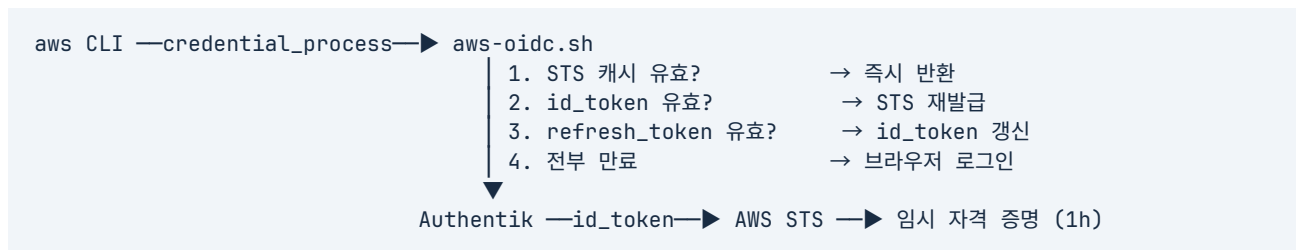
경로	호출자	IP 제한
로그인 UI, authorize	사용자 브라우저	적용
token, userinfo, jwks, discovery	앱 pod (서버 간 호출)	미적용. 출발지가 사용자 IP가 아님
AWS CLI용 provider의 jwks, discovery	AWS STS	미적용. AWS가 토큰 서명을 검증하러 올

IngressRoute 하나 안에서 경로와 priority로 나눈다. 사람이 쓰는 화면은 막고, 기계가 서명을 확인하는 공개 endpoint만 연다.

6. AWS 접근: 장기 자격 증명을 없애기

대상	이전	현재
콘솔	IAM User + 비밀번호	Authentik이 SAML IdP, IAM이 SP. 세션은 임시 자격 증명
CLI	access key	OIDC + PKCE → AssumeRoleWithWebIdentity → 임시 자격 증명

CLI: credential_process로 투명하게



aws-oidc.sh는 AWS CLI가 자격 증명이 필요할 때 호출하는 프로그램이다. 사용자는 하루에 한 번 브라우저 로그인을 하고, 그 뒤 12시간은 refresh token이 조용히 갱신한다. aws를 호출하는 다른 도구와 에이전트도 같은 프로필을 그대로 쓴다.

콘솔용 SAML과 CLI용 OIDC를 나눈 이유는 이 구현에서의 갱신 방식이다. CLI는 refresh token으로 OIDC 토큰을 갱신하고 STS 자격 증명을 재발급하도록 설계해, 반복 호출 시 브라우저 재인증 빈도를 줄였다.

IAM 쪽 신뢰 조건

authentik-aws-oidc 모듈의 관련 필드 발췌:

```

Action      = "sts:AssumeRoleWithWebIdentity"
Principal = { Federated = aws_iam_openid_connect_provider.authentik.arn }
Condition = {
  StringEquals = { "<idp-host>:aud" = var.oidc_client_ids[0] } # 이 client 가 발급한 토큰만
  StringLike   = { "<idp-host>:sub" = var.allowed_sub_pattern } # 회사 도메인 계정만
  # allowed_source_ips 가 있으면 IPAddress 조건 추가
}

```

sub를 조건으로 쓰기 위해 provider의 sub_mode를 이메일로 두었다. IAM OIDC 신뢰 정책이 조건으로 쓸 수 있는 claim은 제한적이라, 토큰의 어느 필드에 무엇을 담을지는 IdP 쪽 설정과 함께 정해야 한다. IAM OIDC provider와 role은 Terraform이, Authentik 쪽 provider는 Blueprint가 선언한다. 양쪽이 한 저장소에 있어서 client ID 같은 연결 값을 한 PR에서 맞출 수 있다.

7. 겪은 함정 모음

증상	원인	해결
Argo CD CLI 로그인 시 토큰 검증 실패	application별 issuer. CLI용 별도 provider는 issuer가 다름	provider 하나를 public + PKCE로 공유 (5-1)
Argo Workflows 무한 리다이렉트	Traefik이 로그인 완료를 알 수 없음	쿠키 유무로 라우트 분기 (5-3)
위 분기가 작동하지 않고 당시 확인한 로그로 원인 파악이 어려웠음	Traefik v3에 v2 matcher 이름 사용	HeaderRegexp로 수정
Argo Workflows code:7 "not allowed"	Kubernetes 1.24부터 ServiceAccount token Secret이 자동 생성되지 않음	kubernetes.io/service-account-token Secret을 직접 선언
oauth2-proxy 재배포 뒤 로그인 루프	당시 args로 전달하던 cookie secret 구성을 점검	네이티브 환경 변수 전달로 통일해 해결 (5-2)
oauth2-proxy --cookie-refresh 사용 시 루프	offline_access scope가 없어 refresh token 미발급	scope 추가
IdP가 산발적으로 503	server CPU limit이 낮아 throttle → readiness 실패 → 서비스 대상에서 제외	CPU limit 상향, replica 2 + 노드 분산 (분산 설계)

마지막 행은 신규 로그인 경로가 IdP에 의존함을 보여준다. 기존 앱 세션은 별도로 유지될 수 있다. [readiness 실패](#) 자체는 컨테이너를 재시작하지 않으므로, 재시작 현상은 liveness·프로세스 종료 등 별도 원인과 구분한다.

8. 한계: Synced가 "적용됨"은 아니다

Blueprint 방식의 가장 큰 함정은 운영 중에 발견했다.

관찰	의미
Argo CD는 Synced + Healthy	ConfigMap이 클러스터에 있다는 뜻일 뿐
Authentik 내부의 Blueprint 상태는 error	서버가 그 YAML을 적용하다 실패

관찰	의미
실패해도 알림이 없음	한 Blueprint가 약 6개월간 적용되지 않은 채 아무도 몰랐음

원인은 URL을 검증하는 필드에 URL이 아닌 값을 넣은 한 줄이었다. Blueprint는 파일 단위로 전부 아니면 전무라서, 그 한 줄 때문에 같은 파일의 다른 변경도 함께 적용되지 않았다. Git에는 의도가 있고 리뷰도 통과했지만 실제 시스템은 그 이전 상태였다.

여기서 확인한 Blueprint의 성질:

성질	결과
파일 단위 원자성	항목 하나의 검증 실패가 파일 전체를 막음
정기 적용과 파일 변경 감시	운영 당시 주석 변경을 계기로 재적용 오류가 드러남. 세부 재적용 동작은 사용 버전과 상태를 확인
기본 실행 순서는 보장되지 않음	객체 의존성은 같은 파일에 두거나 metaapplyblueprint로 명시
선언에서 지운 객체는 DB에 남음	삭제는 state: absent로 명시해야 함

이 적용 방식과 원자성·순서 제약은 공식 Blueprint 문서를 기준으로 구분했다. Argo CD의 기본 상태 확인만으로는 애플리케이션 내부 적용 성공을 알 수 없어, Blueprint 상태를 직접 조회하는 점검을 검증 절차에 넣었다. 자동 감시는 아직 과제다.

그 밖에 코드로 옮기지 못한 것:

항목	이유
로그인 화면에 Google source 연결	당시 구성에서 list 내부 !Find 참조를 해결하지 못해 수동 연결 유지
사용자의 그룹 배정	사람에 대한 판단이라 의도적으로 수동

9. 결과

항목	이전	현재
입사자 온보딩	도구마다 계정 생성	Google 계정 로그인 + 그룹 추가 한 번
퇴사자 처리	도구마다 계정 삭제	신규 인증 차단은 IdP로 중앙화. 기존 세션·임시 자격 증명은 별도 만료·회수 확인
인증 없는 도구	IP 제한만	proxy 뒤에서 SSO
AWS 콘솔	IAM User (장기)	SAML (임시)
AWS CLI	access key (장기)	OIDC + credential_process (임시, 자동 갱신)
새 도구 추가	계정 체계를 새로 설계	Blueprint에 application 항목 추가

가장 큰 변화는 편의가 아니라 사람에게 발급된 장기 자격 증명에 없어진 것이다.

서비스 메시를 걷어내기: Istio에서 Traefik으로

서비스 메시를 도입하는 글은 많고 걷어내는 글은 적다. 이 문서는 운영 중인 EKS 클러스터에서 Istio를 제거하고 Traefik으로 옮긴 판단과 과정을 설명한다. 핵심은 도구 교체가 아니라 **"우리가 실제로 쓰고 있는 기능이 무엇인가"**를 켜는 것이다.

근거	대상	기준
전환 전	manifest-k8s-cluster-portfolio	commit ed4f0a3
전환 후	devops-configs-portfolio	commit 7e3427a

두 스냅샷의 설정을 비교한 문서이며, 운영 클러스터의 현재 상태를 증명하지는 않는다. 전환 전 스냅샷은 일부 서비스만 담고 있다. 저장소 구조는 [GitOps 저장소 아키텍처](#)와 [공통화 설계](#)에서 다룬다.

1. 문제: 메시 값을 내고 인그레스만 쓰고 있었다

불편은 두 쪽에서 왔다.

누가	무엇이 힘들었나
개발자	요청이 어디서 막혔는지 찾기 어려움. 구조를 이해하려면 Istio부터 알아야 함. 새로 온 사람에게 인수인계가 어려움
플랫폼 엔지니어	pod마다 붙는 sidecar proxy 관리, Istio 고유의 설정 체계, 그리고 문제가 생기면 결국 Envoy까지 내려가야 하는 디버깅

학습해야 할 층이 세 겹이었다.

Kubernetes → Istio (Gateway · VirtualService · 주입 규칙) → Envoy (실제 동작과 로그)

Envoy 수준까지 익히는 일은 우선순위가 될 수 없었다. 플랫폼을 운영하는 사람이 적었고, 같은 시간에 해야 할 일이 더 많았다.

그래서 **무엇을 쓰고 있는지를** 켜었다. 전환 전 스냅샷의 Istio 리소스:

리소스	수	역할
Gateway	4	인그레스 호스트 선언
VirtualService	4	경로 매칭, rewrite, 목적지 지정
DestinationRule, PeerAuthentication, AuthorizationPolicy, EnvoyFilter 등	0	트래픽 정책, mTLS, 서비스 간 인가

공개 스냅샷에서 확인되는 명시적 설정은 L7 인그레스 라우팅 중심이었다. VirtualService template은 prefix를 매칭해 /로 rewrite하고 서비스로 보낸다. 다만 정책이 없어도 sidecar 간 auto mTLS는 동작할 수 있으므로, 일부 스냅샷의 리소스 수만으

로 전체 메시 기능의 미사용을 단정할 수는 없다.

반대로 비용은 전부 내고 있었다.

비용	내용
pod당 고정 오버헤드	주입 대상 namespace의 모든 pod에 proxy 컨테이너가 하나씩 붙음. 서비스가 한가해도 요청값은 그대로
제어면	istiod와 ingress gateway를 별도로 운영하고 업그레이드해야 함
예외 처리	배치 Job은 sidecar가 끝나지 않아 완료되지 못하므로 sidecar.istio.io/inject: "false"를 따로 달아야 했음
인지 부하	위의 세 겹

2. 판단: 메시가 필요한 조건인가

메시가 값을 하는 조건	이 환경
서비스 간 호출이 많고 그 그래프가 복잡함	서비스 20여 개. MSA 경계가 뚜렷하지 않고 대부분의 트래픽이 외부 → 서비스 방향
서비스 간 mTLS와 인가를 정책으로 강제해야 함	공개 스냅샷에서 명시적 강제 정책은 확인되지 않음
카나리·미러링 같은 세밀한 트래픽 제어를 일상적으로 씀	사용하지 않음
메시를 전담할 사람이 있음	없음

팀이 전환 당시 운영 요구로 정리한 것은 인그레스 기능이었다. 남은 질문은 "메시 없이 필요한 기능을 제공할 수 있는가"였고, 목록은 짧았다.

실제로 필요한 것	근거
host·path 라우팅과 우선순위	한 호스트 아래 /seller/, /admin/, /가 다른 서비스로 감
path prefix 제거	기존 rewrite: uri: /
CORS 헤더	브라우저에서 직접 호출하는 API
IP allowlist	사내 도구
rate limit, retry, 압축	공통 middleware

비용을 줄여야 하는 시기였다는 점도 같은 방향을 가리켰다. 쓰지 않는 기능을 위해 pod마다 자원을 예약해 둘 이유가 없었다.

3. 대안 비교: 왜 Traefik인가

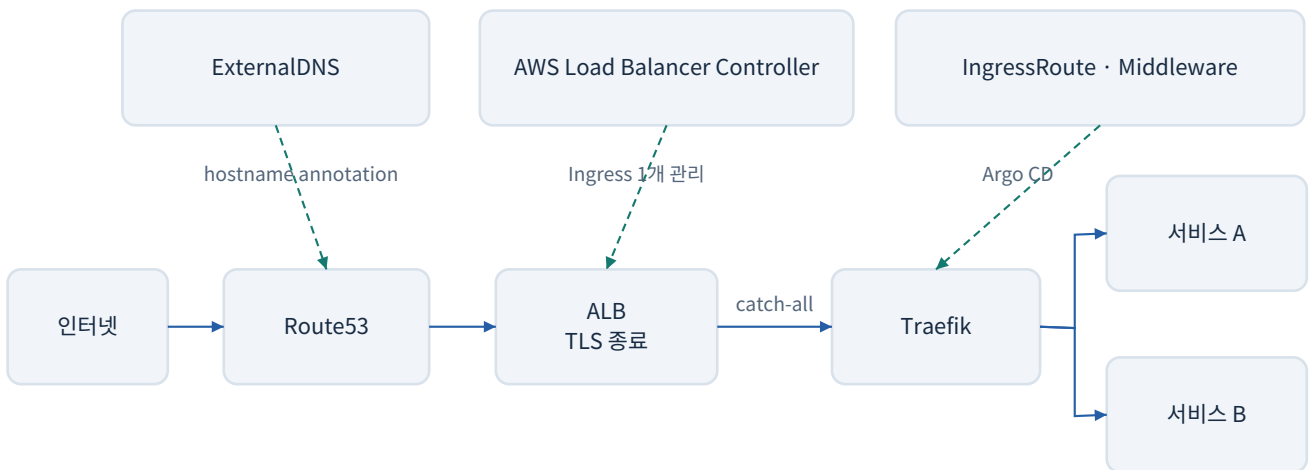
선택지	판단
Istio 유지	1~2절의 문제가 그대로 남음
ALB 단독 (AWS Load Balancer Controller의 Ingress만 사용)	URL rewrite·CORS 응답 헤더도 지원. 이 환경에서는 서비스 라우팅과 공통 middleware를 Kubernetes CRD로 관리하고 AWS 리소스 변경과 분리하는 방식을 선호
ingress-nginx	사실상의 표준이지만 retirement가 발표됨 . 2026년 3월까지 best-effort 유지보수, 그 뒤로는 릴리스와 보안 패치가 없음 (Kubernetes 공지)
Traefik	필요한 기능을 IngressRoute와 Middleware 두 CRD로 선언. 유지보수가 활발함

가장 익숙한 선택지는 ingress-nginx였다. 하지만 복잡도를 줄이려고 옮기는 마당에, 옮기자마자 다시 옮겨야 할 도구를 고를 수는 없었다. 이 전환의 시점(2025년 12월)이 공지 직후였다는 점이 결정을 단순하게 만들었다. 도구를 고를 때 기능표만큼 **그 도구**가 **3년 뒤에도 패치되는가**를 본다.

Traefik을 고른 두 번째 이유는 설정의 모양이다. 라우팅 규칙이 annotation 문자열이 아니라 CRD 필드라서 Git diff로 읽히고, 재사용할 가공 단계는 Middleware로 떼어 여러 라우트가 참조한다. ALB의 [URL rewrite](#)와 [CORS 응답 헤더 삽입](#)도 전환 시점에 제공되던 기능이므로, 차이는 기능의 유무보다 관리 방식과 책임 분리에 있다.

4. 설계: AWS 리소스와 애플리케이션 라우팅을 분리

Traefik만 넣은 것이 아니라 누가 무엇을 소유하는지를 나눴다.



층	소유자	맡는 것	바뀌는 때
AWS 리소스	AWS Load Balancer Controller + ExternalDNS	ALB, listener, ACM 인증서, health check, access log, DNS 레코드	도메인·인증서가 추가될 때

층	소유자	맡는 것	바뀌는 때
애플리케이션 라우팅	Traefik	host·path 매칭, prefix 제거, CORS, allowlist, rate limit	서비스가 추가되거나 경로가 바뀔 때

연결점은 Ingress 하나다. 관련 필드 발췌:

```
kind: Ingress
metadata:
  annotations:
    alb.ingress.kubernetes.io/target-type: ip          # pod IP 를 직접 타깃으로
    alb.ingress.kubernetes.io/ssl-redirect: "443"
    alb.ingress.kubernetes.io/healthcheck-path: /ping # Traefik 의 health endpoint
    alb.ingress.kubernetes.io/healthcheck-port: "9000"
spec:
  ingressClassName: alb
  rules:
    - http:
        paths:
          - path: /                                # 전부 Traefik 으로
            backend: { service: { name: traefik, port: { number: 80 } } }
```

ALB는 규칙을 하나만 가진다. 서비스가 늘어도 AWS 쪽은 변하지 않고, 인증서를 바꿔도 라우팅은 건드리지 않는다. 두 종류의 변경이 서로의 리뷰와 장애 환경에 들어오지 않는다.

계획 단계와 달라진 점이 하나 있다. **마이그레이션 계획**은 Terraform으로 만든 ALB가 NodePort로 Traefik에 붙는 구조였고, 최종 구조는 Load Balancer Controller가 관리하는 ALB가 pod IP를 직접 타깃으로 삼는다. 노드는 Karpenter가 수시로 교체하므로, 타깃이 노드가 아니라 pod을 따라가는 쪽이 타깃 그룹을 안정적으로 유지한다.

5. 옮기기: VirtualService에서 IngressRoute로

Istio	Traefik
Gateway (hosts, ingressgateway selector)	IngressRoute의 Host() matcher. 별도 리소스 불필요
VirtualService의 match.uri.prefix	PathPrefix() + priority
rewrite: uri: /	Middleware stripPrefix
route.destination	services
(서비스 chart마다 template)	환경별 파일 몇 개에 모음

전환 전, 서비스 chart 안의 template:

```
kind: VirtualService
spec:
  gateways: [<gateway>]
  http:
    - match: [{ uri: { prefix: /admin } }]
      rewrite: { uri: / }
      route: [{ destination: { host: <service>, port: { number: 80 } } }]
```

전환 후, mall.yaml의 관련 필드 발췌:

```
kind: IngressRoute
spec:
  routes:
    - match: Host(`api-v4...`) && PathPrefix(`/admin/`)
      priority: 20
      middlewares: [{ name: strip-admin-prefix }, { name: cors-headers }]
      services: [{ name: mall-v4-api-admin, port: 80 }]
    - match: Host(`api-v4...`) && PathPrefix(`/`)
      priority: 10
      services: [{ name: mall-v4-api, port: 80 }]
```

위치도 옮겼다. 라우팅이 서비스 chart의 template에 흩어져 있을 때는 "이 호스트로 들어온 요청이 어디로 가는가"에 답하려면 chart 여러 개를 열어야 했다. 지금은 manifests/traefik/<env>/ 아래 도메인별 파일에 모여 있다. 스냅샷 기준 IngressRoute 38개, Middleware 29개, prd 호스트 21개다.

같은 경로를 두 곳에서 정의하지 않도록 규칙도 하나 두었다. Traefik이 라우팅하는 서비스는 Helm chart의 ingress를 끈다.

6. 전환: 끊지 않고 옮기기

단계	한 일	되돌리는 방법
0. 병행 준비	새 ALB를 기존 Istio ALB 옆에 세움. 테스트용 도메인만 연결	새 ALB 삭제
1. Traefik 배포	Argo CD로 배포, ALB health check가 /ping 에서 통과하는지 확인	Application 삭제
2. 쉬운 것부터	내부 도구 하나를 테스트 도메인으로 먼저	DNS 레코드 삭제
3. 어려운 것 검증	prefix rewrite와 CORS가 있는 API를 경로별로 확인	//
4. 트래픽 이동	Route53 가중치 레코드로 90:10 → 50:50 → 0:100, 단계 사이 10분 관찰	가중치를 되돌림
5. 관찰	요청 수, 지연, 에러율을 Traefik 메트릭과 access log로 확인	//
6. Istio 제거	안정 기간 뒤 VirtualService·Gateway → ingress gateway → control plane 순으로 삭제	(이 시점부터 롤백 계획 폐기)

핵심은 4단계다. 옛 경로와 새 경로가 **각자의 ALB로 동시에 살아 있고**, 전환은 DNS 가중치 숫자 하나다. 롤백도 같은 숫자를 되돌리는 것이라 새 경로에 문제가 있어도 옛 경로는 손대지 않은 채 그대로다. dns-weighted-migrate.sh가 도메인별로 이 단계를 실행한다.

제거 순서도 의도적이다. 의존하는 쪽부터 지운다. 라우팅 리소스, ingress gateway, 제어면 순이다. 제어면을 먼저 지우면 그것에 의존하는 컴포넌트가 설정을 받지 못한 채 남는 구간이 생긴다.

7. 진입점이 하나가 되었으므로

모든 외부 트래픽이 Traefik을 지난다. 단일 장애점이 되지 않도록 prd values에 가용성 설정을 묶었다.

설정	값
HPA	3 ~ 10
PDB	minAvailable: 2
분산	zone 권고 + hostname 강제, nodeTaintsPolicy: Honor
우선순위	system-cluster-critical
X-Forwarded-For 신뢰	VPC 대역에서 온 요청만

replica 3 이상에서 Honor가 빠지면 세 번째 pod이 Pending에 걸린다. 실제로 Traefik에 강제 분산을 처음 적용했을 때 이 일을 겪었다. 원리와 재현은 [EKS 워크로드 분산 설계 3-1절](#)에 있다.

8. 결과

항목	전환 전	전환 후
pod 구성	앱 + sidecar proxy	앱만
상시 운영 컴포넌트	istiod, ingress gateway, 모든 pod의 sidecar	Traefik Deployment 하나
구조를 이해하려면	Kubernetes + Istio + Envoy	Kubernetes + CRD 두 종류
"이 요청이 어디로 가나"	서비스 chart들의 template	환경별 라우팅 파일
디버깅	sidecar와 gateway의 Envoy 로그	Traefik JSON access log를 호스트로 필터
새 서비스 노출	chart에 Gateway·VirtualService 값 추가	IngressRoute 추가 + hostname annotation 한 줄
배치 Job	sidecar 주입 예외 처리 필요	불필요

20여 개 서비스를 옮겼다. 자원 측면에서는 제어면과 gateway에 더해 **pod 수에 비례하던 sidecar 요청값**이 사라졌다. Istio의 기본 sidecar 요청값은 CPU 100m, 메모리 128Mi이고, 이것이 서비스 부하와 무관하게 replica마다 예약된다. 실측 절감치는 이 스냅샷에 없다.

가장 크게 바뀐 것은 인수인계다. 새로 온 사람에게 설명할 그림이 "ALB → Traefik → 서비스" 한 줄이 되었다.

9. 잃은 것과 남은 것

항목	상태
서비스 간 mTLS	Traefik이 대체하지 않음. 이전 auto mTLS 사용 여부는 스냅샷으로 확인되지 않으며, 강제 요구가 생기면 별도 설계 필요
서비스 간 호출 관측	메시 텔레메트리 없음. 서비스 간 추적은 APM에 의존
라우트 설정 오류 진단	matcher 이름·Middleware 참조 오류를 당시 로그에서 바로 찾지 못해 라우트 문법과 참조 대상을 직접 확인 (SSO 문서 5-3절)
도메인 목록이 한 곳에	모든 호스트가 Ingress 하나의 annotation에 있어 변경이 한 파일에 몰림. 역할 분리의 대가
Istio 흔적	배치 chart에 disableIstio 옵션 등 쓰이지 않는 설정이 남아 있음
Ingress API의 다음	upstream은 Gateway API를 권장. Traefik도 구현하고 있어 경로는 열려 있으나 아직 옮기지 않음

메시가 틀린 도구였던 것은 아니다. **이 규모와 이 팀에 맞지 않았을 뿐이다.** 조건이 2절의 표에 해당하게 되면 다시 검토할 일이다.